MANUAL TÉCNICO

TRACKER DE ISSUES: CODE ‘N BUGS
PROYECTO 1:
ANÁLISIS Y DISEÑO DE SISTEMAS 2

202131284 - José Roberto Bautista Rojas

Primer Semestre 2026 - Análisis y diseño de sistemas 2

BREVE DESCRIPCIÓN

El presente proyecto, es una implementación que busca digitalizar y centralizar el manejo de casos de errores o mejoras en código de proyectos para una empresa dedicada al desarrollo de software para poder llevar un control de cuanto tiempo se usa para resolver los problemas registrados y tambien el control financiero para saber cuanto se gasta en la resolución de dichos problemas.

OBJETIVOS DE ESTE MANUAL

Objetivo General

Documentar de forma clara y estructurada la arquitectura, tecnologías, infraestructura, testing y configuración del proyecto **Code 'n Bugs Tracker**, con el fin de servir como guía técnica para luego comprender, desplegar, mantener y evaluar la solución implementada en la plataforma AWS.

Objetivos Específicos

- Describir las **herramientas, lenguajes y frameworks** utilizados en el desarrollo del frontend y los servicios backend del sistema.

- Explicar la **arquitectura general del proyecto**, incluyendo el modelo cliente-servidor y la comunicación mediante APIs REST.

- Detallar la **infraestructura desplegada en AWS**, especificando el uso y configuración de servicios como EC2, RDS, S3, Elastic Beanstalk, ECR, y IAM.

- Documentar el **proceso de despliegue continuo (CI/CD)** implementado mediante GitHub Actions y el uso de contenedores Docker.

- Presentar la **estructura de seguridad y control de accesos**, mostrando el rol de IAM, autenticación, autorización y protección de datos.

- Explicar las decisiones técnicas tomadas durante el desarrollo y despliegue del sistema, así como las **limitaciones y soluciones adoptadas** (por ejemplo, el uso de imágenes preconstruidas en ECR).

- Servir como **material de referencia técnico** para docentes, evaluadores y futuros desarrolladores que requieran comprender o extender el sistema.

HERRAMIENTAS Y TECNOLOGÍAS UTILIZADAS

Lenguajes y Frameworks de Desarrollo

Para el desarrollo de la plataforma se utilizaron múltiples tecnologías, seleccionadas según el rol de cada componente dentro de la arquitectura:

- **Backend:**
  Se desarrolló utilizando el framework de Java **Spring Boot** por su enfoque modular, escalable y orientado a arquitecturas empresariales. Para el acceso a datos se utilizó **Hibernate** como ORM, junto con otras librerías complementarias para validación, autenticación y manejo de configuración.

- **Frontend:**
  El frontend fue desarrollado con **Vue.js**, utilizando **Nuxt** para la generación del sitio de forma estática. Para la interfaz de usuario se empleó la librería **shadcn/ui**, junto con **Tailwind CSS** para el diseño y estilización responsive de los componentes. El resultado final es un sitio estático optimizado para despliegue en la nube.

Herramientas de Desarrollo y Control de Versiones

- **Editor de código:** Visual Studio Code

- **Control de versiones:** Git

- **Repositorio remoto:** GitHub

- **Sistema operativo:** macOS versión 15

Base de Datos

- **Sistema gestor:** PostgreSQL
  Utilizado como base de datos relacional principal para el manejo de usuarios, pasos, tipos de caso, proyectos, casos, worklogs y demás entidades del sistema.

Infraestructura y Servicios de Despliegue

El despliegue de la plataforma se realizó completamente sobre **Amazon Web Services (AWS)**, utilizando los siguientes servicios:

- **Amazon RDS:**
  Instancia administrada de PostgreSQL para la persistencia de datos, garantizando alta disponibilidad y seguridad.

- **Amazon EC2:**
  Una instancia EC2 destinada a la ejecución del servicio backend

- **Amazon ECR (Elastic Container Registry):**
  Repositorio de imágenes Docker para almacenar las imágenes del servicio de backend.

- **Elastic Beanstalk:**
  Utilizado para orquestar el despliegue de los contenedores Docker. Se configuró un **environment independiente**, utilizando el preset de Docker.

- **Amazon S3:**

  - Un bucket configurado para **Static Website Hosting**, utilizado para servir el frontend generado de forma estática.

- **IAM (Identity and Access Management):**
  Definición de usuarios, roles y políticas de acceso necesarias para la correcta interacción entre los servicios desplegados.

- **GMail SMTP Server:**
  Servicio utilizado para el envío de correos electrónicos (confirmaciones, recuperación de contraseña, etc.), Se optó por esta opción ya que es más rápida de implementar y no requiere tener un dominio propio.

Arquitectura de Despliegue

La arquitectura implementada sigue un modelo **cliente-servidor**, donde el frontend se comunica con el backend mediante **requests HTTP bajo un enfoque REST**.

El flujo de despliegue es el siguiente:

1. El código fuente se mantiene centralizado en **GitHub**.

2. Se configuraron **GitHub Actions** como pipeline de integración continua.

3. Al realizar un *push* a la rama main:

    - Se construye automáticamente la imágen Docker de backend.

    - La imágen resultante se publica en **Amazon ECR**.

4. **Elastic Beanstalk** obtiene la imágen previamente construida desde ECR y despliega el contenedor directamente en la instancia EC2.

Este enfoque se adoptó debido a las limitaciones de memoria de las instancias EC2 del *free tier*, ya que construir las imágenes Docker directamente en dichas instancias provocaba fallos por consumo excesivo de recursos.

El frontend se genera de forma manual mediante el comando:

```
nuxt generate
```

Los archivos resultantes se suben al bucket de **Amazon S3**, configurado específicamente para servir contenido estático con acceso público.

Conclusiones

- La arquitectura implementada cumple con los lineamientos establecidos en el enunciado del proyecto, garantizando un despliegue completo en la nube y evitando dependencias locales.

- El uso de una arquitectura basada en contenedores facilitó la portabilidad, escalabilidad y mantenimiento de los servicios backend.

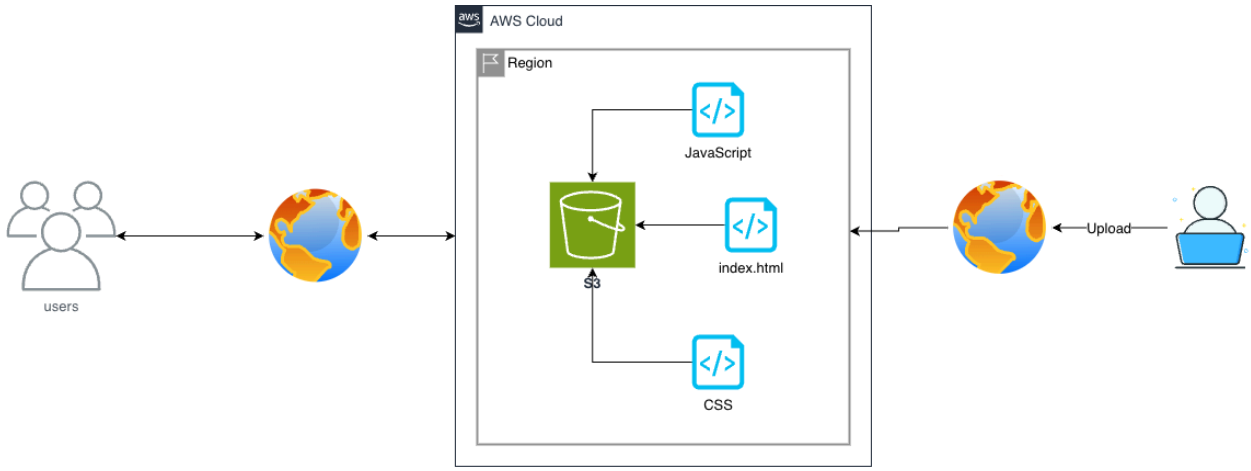
- La separación de responsabilidades entre frontend y backend permitió una mejor organización del proyecto y una comunicación clara mediante APIs REST.

- La integración de pipelines de CI/CD con GitHub Actions redujo errores manuales y optimizó el proceso de despliegue.

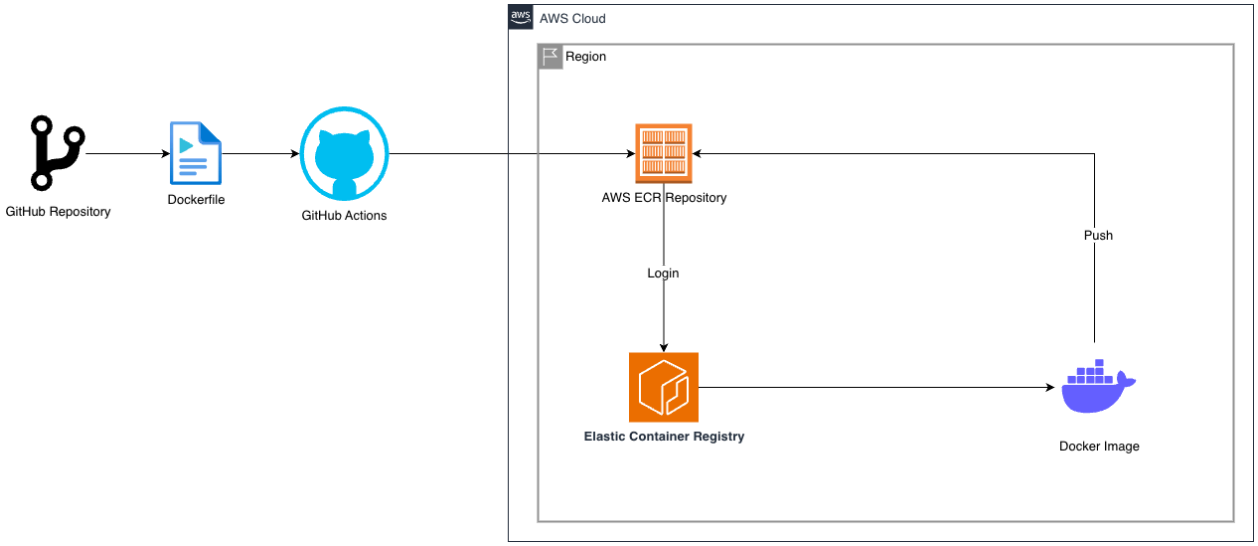
- El uso de servicios administrados de AWS, como RDS y Elastic Beanstalk, simplificó la gestión de infraestructura y mejoró la estabilidad del sistema.

Diagramas de arquitectura de despliegue

Frontend:



Backend - Etapa de Construcción (Build)



Backend - Etapa de Despliegue, con load balancing y servicios externos

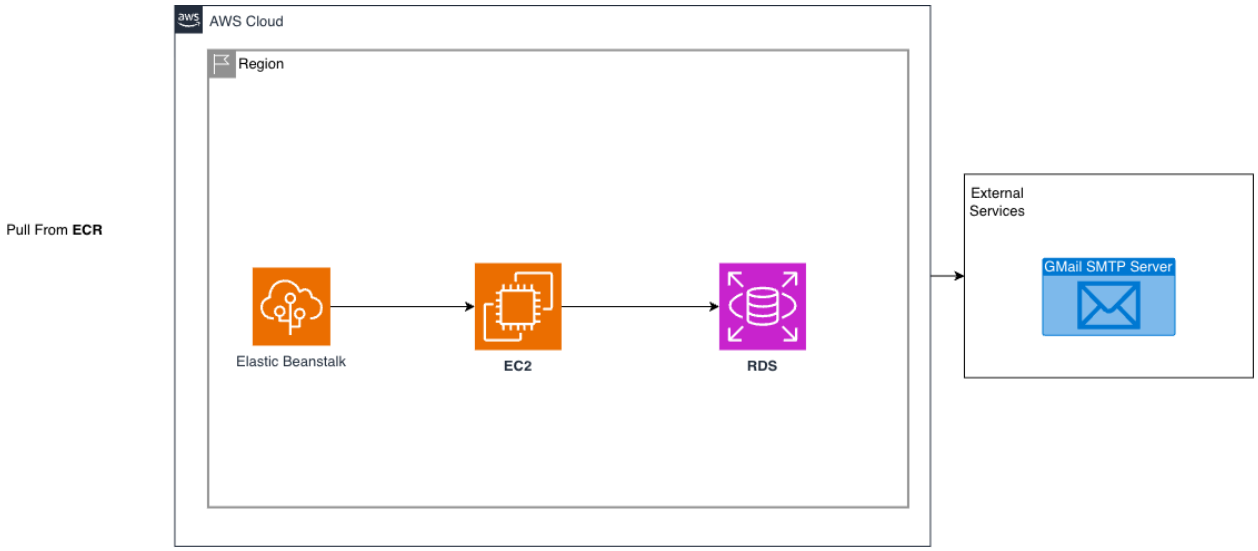
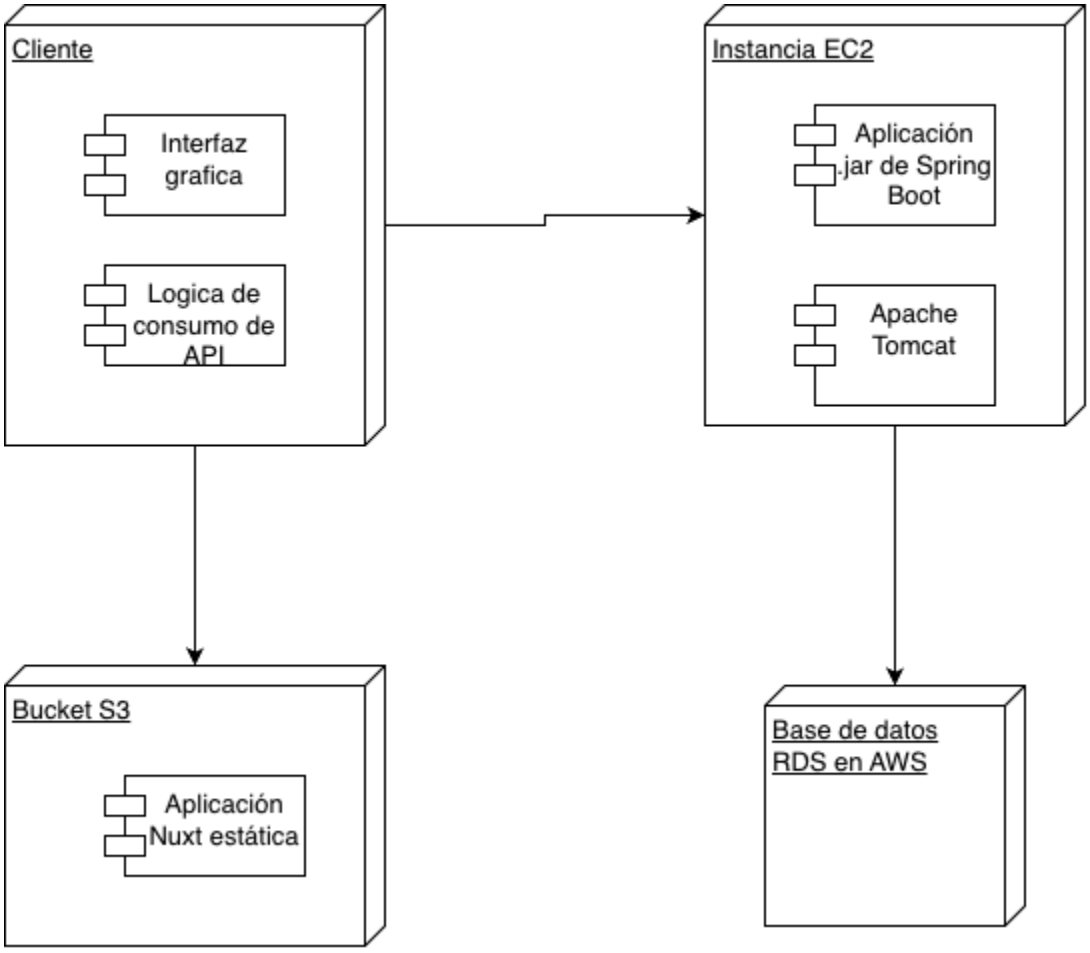


Diagrama de despliegue



Arquitectura Lógica del Sistema

La arquitectura lógica del sistema **Code 'n Bugs Tracker** está diseñada bajo un enfoque **cliente–servidor**, utilizando una separación clara de responsabilidades entre la capa de presentación, la capa de servicios y la capa de datos. Esta arquitectura permite escalabilidad, mantenibilidad y una comunicación estructurada entre los distintos componentes del sistema.

Modelo Cliente–Servidor

El sistema sigue un modelo clásico de arquitectura cliente–servidor, donde el cliente (navegador web) consume los servicios expuestos por el backend a través de **peticiones HTTP** bajo el estándar **REST**. El cliente no accede directamente a la base de datos ni a servicios internos, garantizando así un mayor nivel de seguridad y control.

La comunicación entre cliente y servidor se realiza mediante:

- Métodos HTTP (GET, POST, PUT, DELETE)
- Formato de intercambio de datos **JSON**
- Endpoints REST organizados por módulos funcionales

Capa de Presentación (Frontend)

La capa de presentación está implementada como una **aplicación web estática**, desarrollada con **Vue.js y Nuxt**, y estilizada mediante **Tailwind CSS** y componentes de **shadcn/ui**.

Sus responsabilidades principales incluyen:

- Interacción directa con el usuario.

- Consumo de los servicios REST expuestos por el backend.

- Manejo de navegación, vistas y estados de interfaz.
- Validaciones básicas del lado del cliente.

Al ser un sitio generado de forma estática, el frontend no contiene lógica de negocio crítica ni acceso directo a datos sensibles, limitándose exclusivamente a la presentación y consumo de información.

Capa de Servicio (Backend)

La lógica de negocio del sistema se encuentra distribuida en **un servicio backend**.

Este servicio está orientado a manejar funcionalidades centrales del sistema, tales como:

- Autenticación y autorización de usuarios.
- Gestión de empleados, pacientes y roles.
- Procesos administrativos y validaciones de negocio.

Spring Boot proporciona una arquitectura modular basada en controladores, servicios y proveedores, lo que facilita la escalabilidad y el mantenimiento del sistema.

Capa de Datos

La capa de datos está centralizada en una base de datos relacional **PostgreSQL**, alojada en un servicio administrado. Esta base de datos es responsable de almacenar información crítica del sistema, incluyendo:

- Usuarios, roles y credenciales.

- Categorización de datos como tipos de casos y etapas.
- Registro de casos y worklogs.

El acceso a la base de datos se realiza únicamente desde el backend, utilizando el ORM que permite abstracción, validación y control de transacciones.

Mapeo físico de la base de datos

```
--
-- Name: audit_logs; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.audit_logs (
    id bigint NOT NULL,
    entity_type character varying(40) NOT NULL,
    entity_id bigint NOT NULL,
    action character varying(60) NOT NULL,
    performed_by_user_id bigint,
    detail jsonb,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL
);


ALTER TABLE public.audit_logs OWNER TO postgres;

--
-- Name: audit_logs_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.audit_logs_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.audit_logs_id_seq OWNER TO postgres;

--
-- Name: audit_logs_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--

ALTER SEQUENCE public.audit_logs_id_seq OWNED BY public.audit_logs.id;


--
-- Name: case_steps; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.case_steps (
    id bigint NOT NULL,
    case_id bigint NOT NULL,
    case_type_stage_id bigint NOT NULL,
    step_order integer NOT NULL,
    status character varying(30) DEFAULT 'PENDING'::character varying NOT NULL,
    assigned_employee_id bigint,
```

```sql
    assigned_at timestamp with time zone,
    started_at timestamp with time zone,
    submitted_at timestamp with time zone,
    approved_at timestamp with time zone,
    rejected_at timestamp with time zone,
    rejection_reason text,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL,
    CONSTRAINT cs_approve_chk CHECK ((((status)::text <> 'APPROVED'::text) OR (approved_at IS NOT NULL))),
    CONSTRAINT cs_reject_chk CHECK ((((status)::text <> 'REJECTED'::text) OR ((rejected_at IS NOT NULL) AND
(rejection_reason IS NOT NULL) AND (length(TRIM(BOTH FROM rejection_reason)) > 0)))),
    CONSTRAINT cs_status_chk CHECK (((status)::text = ANY ((ARRAY['PENDING'::character varying, 'ASSIGNED'::character
varying, 'IN_PROGRESS'::character varying, 'SUBMITTED'::character varying, 'APPROVED'::character varying,
'REJECTED'::character varying])::text[]))),
    CONSTRAINT cs_step_order_chk CHECK ((step_order >= 1))
);


ALTER TABLE public.case_steps OWNER TO postgres;

--
-- Name: case_steps_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.case_steps_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.case_steps_id_seq OWNER TO postgres;

--
-- Name: case_steps_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--

ALTER SEQUENCE public.case_steps_id_seq OWNED BY public.case_steps.id;


--
-- Name: case_tickets; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.case_tickets (
    id bigint NOT NULL,
    project_id bigint NOT NULL,
    case_type_id bigint NOT NULL,
    created_by_employee_id bigint NOT NULL,
    title character varying(250) NOT NULL,
    description text,
    status character varying(30) DEFAULT 'OPEN'::character varying NOT NULL,
    due_date date NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL,
    canceled_at timestamp with time zone,
    cancel_reason text,
    CONSTRAINT case_cancel_chk CHECK ((((status)::text <> 'CANCELED'::text) OR ((canceled_at IS NOT NULL) AND
(cancel_reason IS NOT NULL) AND (length(TRIM(BOTH FROM cancel_reason)) > 0)))),
    CONSTRAINT case_status_chk CHECK (((status)::text = ANY ((ARRAY['OPEN'::character varying,
'IN_PROGRESS'::character varying, 'COMPLETED'::character varying, 'CANCELED'::character varying])::text[])))
);


ALTER TABLE public.case_tickets OWNER TO postgres;
```

```sql
--
-- Name: case_tickets_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.case_tickets_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.case_tickets_id_seq OWNER TO postgres;

--
-- Name: case_tickets_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--

ALTER SEQUENCE public.case_tickets_id_seq OWNED BY public.case_tickets.id;


--
-- Name: case_type_stages; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.case_type_stages (
    id bigint NOT NULL,
    case_type_id bigint NOT NULL,
    name character varying(150) NOT NULL,
    description text,
    stage_order integer NOT NULL,
    active boolean DEFAULT true NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL,
    CONSTRAINT cts_stage_order_chk CHECK ((stage_order >= 1))
);


ALTER TABLE public.case_type_stages OWNER TO postgres;

--
-- Name: case_type_stages_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.case_type_stages_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.case_type_stages_id_seq OWNER TO postgres;

--
-- Name: case_type_stages_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--

ALTER SEQUENCE public.case_type_stages_id_seq OWNED BY public.case_type_stages.id;


--
-- Name: case_types; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.case_types (
    id bigint NOT NULL,
```

```sql
    name character varying(150) NOT NULL,
    description text,
    active boolean DEFAULT true NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL
);


ALTER TABLE public.case_types OWNER TO postgres;

--
-- Name: case_types_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.case_types_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.case_types_id_seq OWNER TO postgres;

--
-- Name: case_types_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--

ALTER SEQUENCE public.case_types_id_seq OWNED BY public.case_types.id;


--
-- Name: employees; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.employees (
    id bigint NOT NULL,
    user_id bigint,
    first_name character varying(120) NOT NULL,
    last_name character varying(120) NOT NULL,
    hourly_rate numeric(12,2) DEFAULT 0 NOT NULL,
    active boolean DEFAULT true NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL,
    CONSTRAINT employee_hourly_rate_nonneg CHECK ((hourly_rate >= (0)::numeric))
);


ALTER TABLE public.employees OWNER TO postgres;

--
-- Name: employees_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.employees_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.employees_id_seq OWNER TO postgres;

--
-- Name: employees_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--
```

```sql
ALTER SEQUENCE public.employees_id_seq OWNED BY public.employees.id;


--
-- Name: flyway_schema_history; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.flyway_schema_history (
    installed_rank integer NOT NULL,
    version character varying(50),
    description character varying(200) NOT NULL,
    type character varying(20) NOT NULL,
    script character varying(1000) NOT NULL,
    checksum integer,
    installed_by character varying(100) NOT NULL,
    installed_on timestamp without time zone DEFAULT now() NOT NULL,
    execution_time integer NOT NULL,
    success boolean NOT NULL
);


ALTER TABLE public.flyway_schema_history OWNER TO postgres;

--
-- Name: onboarding_tokens; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.onboarding_tokens (
    id bigint NOT NULL,
    user_id bigint NOT NULL,
    token character varying(64) NOT NULL,
    used boolean DEFAULT false NOT NULL,
    expires_at timestamp with time zone NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL
);


ALTER TABLE public.onboarding_tokens OWNER TO postgres;

--
-- Name: onboarding_tokens_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.onboarding_tokens_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.onboarding_tokens_id_seq OWNER TO postgres;

--
-- Name: onboarding_tokens_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--

ALTER SEQUENCE public.onboarding_tokens_id_seq OWNED BY public.onboarding_tokens.id;


--
-- Name: password_reset_tokens; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.password_reset_tokens (
```

```sql
    id bigint NOT NULL,
    user_id bigint NOT NULL,
    token character varying(64) NOT NULL,
    used boolean DEFAULT false NOT NULL,
    expires_at timestamp with time zone NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL
);


ALTER TABLE public.password_reset_tokens OWNER TO postgres;

--
-- Name: password_reset_tokens_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.password_reset_tokens_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.password_reset_tokens_id_seq OWNER TO postgres;

--
-- Name: password_reset_tokens_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--

ALTER SEQUENCE public.password_reset_tokens_id_seq OWNED BY public.password_reset_tokens.id;


--
-- Name: project_admin_assignment; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.project_admin_assignment (
    id bigint NOT NULL,
    project_id bigint NOT NULL,
    employee_id bigint NOT NULL,
    start_date date DEFAULT CURRENT_DATE NOT NULL,
    end_date date,
    active boolean DEFAULT true NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL,
    CONSTRAINT paa_dates_chk CHECK (((end_date IS NULL) OR (end_date >= start_date)))
);


ALTER TABLE public.project_admin_assignment OWNER TO postgres;

--
-- Name: project_admin_assignment_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.project_admin_assignment_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.project_admin_assignment_id_seq OWNER TO postgres;

--
```

```sql
-- Name: project_admin_assignment_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--

ALTER SEQUENCE public.project_admin_assignment_id_seq OWNED BY public.project_admin_assignment.id;


--
-- Name: projects; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.projects (
    id bigint NOT NULL,
    name character varying(200) NOT NULL,
    description text,
    status character varying(20) DEFAULT 'ACTIVE'::character varying NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL,
    CONSTRAINT project_status_chk CHECK (((status)::text = ANY ((ARRAY['ACTIVE'::character varying,
'INACTIVE'::character varying])::text[])))
);


ALTER TABLE public.projects OWNER TO postgres;

--
-- Name: projects_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.projects_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.projects_id_seq OWNER TO postgres;

--
-- Name: projects_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--

ALTER SEQUENCE public.projects_id_seq OWNED BY public.projects.id;


--
-- Name: roles; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.roles (
    id bigint NOT NULL,
    code character varying(50) NOT NULL,
    name character varying(100) NOT NULL,
    description text,
    active boolean DEFAULT true NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL
);


ALTER TABLE public.roles OWNER TO postgres;

--
-- Name: roles_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.roles_id_seq
```

```sql
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.roles_id_seq OWNER TO postgres;

--
-- Name: roles_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--

ALTER SEQUENCE public.roles_id_seq OWNED BY public.roles.id;


--
-- Name: users; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.users (
    id bigint NOT NULL,
    username character varying(100) NOT NULL,
    email character varying(200) NOT NULL,
    password_hash text NOT NULL,
    active boolean DEFAULT true NOT NULL,
    role_id bigint NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    updated_at timestamp with time zone DEFAULT now() NOT NULL,
    onboarding_completed boolean DEFAULT false NOT NULL
);


ALTER TABLE public.users OWNER TO postgres;

--
-- Name: users_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.users_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.users_id_seq OWNER TO postgres;

--
-- Name: users_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--

ALTER SEQUENCE public.users_id_seq OWNED BY public.users.id;


--
-- Name: work_logs; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.work_logs (
    id bigint NOT NULL,
    case_step_id bigint NOT NULL,
    employee_id bigint NOT NULL,
    comment text NOT NULL,
    hours_spent numeric(10,2) NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
```

```sql
    updated_at timestamp with time zone DEFAULT now() NOT NULL,
    CONSTRAINT wl_comment_chk CHECK ((length(TRIM(BOTH FROM comment)) > 0)),
    CONSTRAINT wl_hours_chk CHECK ((hours_spent > (0)::numeric))
);


ALTER TABLE public.work_logs OWNER TO postgres;

--
-- Name: work_logs_id_seq; Type: SEQUENCE; Schema: public; Owner: postgres
--

CREATE SEQUENCE public.work_logs_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.work_logs_id_seq OWNER TO postgres;

--
-- Name: work_logs_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: postgres
--

ALTER SEQUENCE public.work_logs_id_seq OWNED BY public.work_logs.id;


--
-- Name: audit_logs id; Type: DEFAULT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.audit_logs ALTER COLUMN id SET DEFAULT nextval('public.audit_logs_id_seq'::regclass);


--
-- Name: case_steps id; Type: DEFAULT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_steps ALTER COLUMN id SET DEFAULT nextval('public.case_steps_id_seq'::regclass);


--
-- Name: case_tickets id; Type: DEFAULT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_tickets ALTER COLUMN id SET DEFAULT nextval('public.case_tickets_id_seq'::regclass);


--
-- Name: case_type_stages id; Type: DEFAULT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_type_stages ALTER COLUMN id SET DEFAULT
nextval('public.case_type_stages_id_seq'::regclass);


--
-- Name: case_types id; Type: DEFAULT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_types ALTER COLUMN id SET DEFAULT nextval('public.case_types_id_seq'::regclass);


--
-- Name: employees id; Type: DEFAULT; Schema: public; Owner: postgres
```

```sql
--

ALTER TABLE ONLY public.employees ALTER COLUMN id SET DEFAULT nextval('public.employees_id_seq'::regclass);


--
-- Name: onboarding_tokens id; Type: DEFAULT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.onboarding_tokens ALTER COLUMN id SET DEFAULT
nextval('public.onboarding_tokens_id_seq'::regclass);


--
-- Name: password_reset_tokens id; Type: DEFAULT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.password_reset_tokens ALTER COLUMN id SET DEFAULT
nextval('public.password_reset_tokens_id_seq'::regclass);


--
-- Name: project_admin_assignment id; Type: DEFAULT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.project_admin_assignment ALTER COLUMN id SET DEFAULT
nextval('public.project_admin_assignment_id_seq'::regclass);


--
-- Name: projects id; Type: DEFAULT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.projects ALTER COLUMN id SET DEFAULT nextval('public.projects_id_seq'::regclass);


--
-- Name: roles id; Type: DEFAULT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.roles ALTER COLUMN id SET DEFAULT nextval('public.roles_id_seq'::regclass);


--
-- Name: users id; Type: DEFAULT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.users ALTER COLUMN id SET DEFAULT nextval('public.users_id_seq'::regclass);


--
-- Name: work_logs id; Type: DEFAULT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.work_logs ALTER COLUMN id SET DEFAULT nextval('public.work_logs_id_seq'::regclass);


--
-- Name: audit_logs audit_logs_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.audit_logs
    ADD CONSTRAINT audit_logs_pkey PRIMARY KEY (id);


--
-- Name: case_steps case_steps_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
```

```sql
--

ALTER TABLE ONLY public.case_steps
    ADD CONSTRAINT case_steps_pkey PRIMARY KEY (id);


--
-- Name: case_tickets case_tickets_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_tickets
    ADD CONSTRAINT case_tickets_pkey PRIMARY KEY (id);


--
-- Name: case_type_stages case_type_stages_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_type_stages
    ADD CONSTRAINT case_type_stages_pkey PRIMARY KEY (id);


--
-- Name: case_types case_types_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_types
    ADD CONSTRAINT case_types_pkey PRIMARY KEY (id);


--
-- Name: employees employees_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.employees
    ADD CONSTRAINT employees_pkey PRIMARY KEY (id);


--
-- Name: employees employees_user_id_key; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.employees
    ADD CONSTRAINT employees_user_id_key UNIQUE (user_id);


--
-- Name: flyway_schema_history flyway_schema_history_pk; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.flyway_schema_history
    ADD CONSTRAINT flyway_schema_history_pk PRIMARY KEY (installed_rank);


--
-- Name: onboarding_tokens onboarding_tokens_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.onboarding_tokens
    ADD CONSTRAINT onboarding_tokens_pkey PRIMARY KEY (id);


--
-- Name: onboarding_tokens onboarding_tokens_token_key; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.onboarding_tokens
```

```sql
    ADD CONSTRAINT onboarding_tokens_token_key UNIQUE (token);


--
-- Name: password_reset_tokens password_reset_tokens_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.password_reset_tokens
    ADD CONSTRAINT password_reset_tokens_pkey PRIMARY KEY (id);


--
-- Name: password_reset_tokens password_reset_tokens_token_key; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.password_reset_tokens
    ADD CONSTRAINT password_reset_tokens_token_key UNIQUE (token);


--
-- Name: project_admin_assignment project_admin_assignment_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.project_admin_assignment
    ADD CONSTRAINT project_admin_assignment_pkey PRIMARY KEY (id);


--
-- Name: projects projects_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.projects
    ADD CONSTRAINT projects_pkey PRIMARY KEY (id);


--
-- Name: roles roles_code_key; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.roles
    ADD CONSTRAINT roles_code_key UNIQUE (code);


--
-- Name: roles roles_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.roles
    ADD CONSTRAINT roles_pkey PRIMARY KEY (id);


--
-- Name: users users_email_key; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.users
    ADD CONSTRAINT users_email_key UNIQUE (email);


--
-- Name: users users_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.users
    ADD CONSTRAINT users_pkey PRIMARY KEY (id);
```

```sql
--
-- Name: users users_username_key; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.users
    ADD CONSTRAINT users_username_key UNIQUE (username);


--
-- Name: work_logs work_logs_pkey; Type: CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.work_logs
    ADD CONSTRAINT work_logs_pkey PRIMARY KEY (id);


--
-- Name: flyway_schema_history_s_idx; Type: INDEX; Schema: public; Owner: postgres
--

CREATE INDEX flyway_schema_history_s_idx ON public.flyway_schema_history USING btree (success);


--
-- Name: idx_audit_entity; Type: INDEX; Schema: public; Owner: postgres
--

CREATE INDEX idx_audit_entity ON public.audit_logs USING btree (entity_type, entity_id, created_at);


--
-- Name: idx_case_step_assigned_status; Type: INDEX; Schema: public; Owner: postgres
--

CREATE INDEX idx_case_step_assigned_status ON public.case_steps USING btree (assigned_employee_id, status);


--
-- Name: idx_case_step_case; Type: INDEX; Schema: public; Owner: postgres
--

CREATE INDEX idx_case_step_case ON public.case_steps USING btree (case_id);


--
-- Name: idx_case_ticket_project_due; Type: INDEX; Schema: public; Owner: postgres
--

CREATE INDEX idx_case_ticket_project_due ON public.case_tickets USING btree (project_id, due_date);


--
-- Name: idx_case_ticket_project_status_due; Type: INDEX; Schema: public; Owner: postgres
--

CREATE INDEX idx_case_ticket_project_status_due ON public.case_tickets USING btree (project_id, status, due_date);


--
-- Name: idx_onboarding_token_token; Type: INDEX; Schema: public; Owner: postgres
--

CREATE INDEX idx_onboarding_token_token ON public.onboarding_tokens USING btree (token);


--
-- Name: idx_onboarding_token_user; Type: INDEX; Schema: public; Owner: postgres
```

```sql
--

CREATE INDEX idx_onboarding_token_user ON public.onboarding_tokens USING btree (user_id);


--
-- Name: idx_password_reset_token_token; Type: INDEX; Schema: public; Owner: postgres
--

CREATE INDEX idx_password_reset_token_token ON public.password_reset_tokens USING btree (token);


--
-- Name: idx_password_reset_token_user; Type: INDEX; Schema: public; Owner: postgres
--

CREATE INDEX idx_password_reset_token_user ON public.password_reset_tokens USING btree (user_id);


--
-- Name: idx_work_log_employee_created; Type: INDEX; Schema: public; Owner: postgres
--

CREATE INDEX idx_work_log_employee_created ON public.work_logs USING btree (employee_id, created_at);


--
-- Name: idx_work_log_step; Type: INDEX; Schema: public; Owner: postgres
--

CREATE INDEX idx_work_log_step ON public.work_logs USING btree (case_step_id);


--
-- Name: uq_case_step_order; Type: INDEX; Schema: public; Owner: postgres
--

CREATE UNIQUE INDEX uq_case_step_order ON public.case_steps USING btree (case_id, step_order);


--
-- Name: uq_case_type_stage_order; Type: INDEX; Schema: public; Owner: postgres
--

CREATE UNIQUE INDEX uq_case_type_stage_order ON public.case_type_stages USING btree (case_type_id, stage_order);


--
-- Name: uq_project_admin_active; Type: INDEX; Schema: public; Owner: postgres
--

CREATE UNIQUE INDEX uq_project_admin_active ON public.project_admin_assignment USING btree (project_id) WHERE (active
= true);


--
-- Name: audit_logs audit_logs_performed_by_user_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.audit_logs
    ADD CONSTRAINT audit_logs_performed_by_user_id_fkey FOREIGN KEY (performed_by_user_id) REFERENCES
public.users(id);


--
-- Name: case_steps case_steps_assigned_employee_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
--
```

```sql
ALTER TABLE ONLY public.case_steps
    ADD CONSTRAINT case_steps_assigned_employee_id_fkey FOREIGN KEY (assigned_employee_id) REFERENCES
public.employees(id);


--
-- Name: case_steps case_steps_case_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_steps
    ADD CONSTRAINT case_steps_case_id_fkey FOREIGN KEY (case_id) REFERENCES public.case_tickets(id) ON DELETE CASCADE;


--
-- Name: case_steps case_steps_case_type_stage_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_steps
    ADD CONSTRAINT case_steps_case_type_stage_id_fkey FOREIGN KEY (case_type_stage_id) REFERENCES
public.case_type_stages(id);


--
-- Name: case_tickets case_tickets_case_type_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_tickets
    ADD CONSTRAINT case_tickets_case_type_id_fkey FOREIGN KEY (case_type_id) REFERENCES public.case_types(id);


--
-- Name: case_tickets case_tickets_created_by_employee_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_tickets
    ADD CONSTRAINT case_tickets_created_by_employee_id_fkey FOREIGN KEY (created_by_employee_id) REFERENCES
public.employees(id);


--
-- Name: case_tickets case_tickets_project_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_tickets
    ADD CONSTRAINT case_tickets_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(id);


--
-- Name: case_type_stages case_type_stages_case_type_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.case_type_stages
    ADD CONSTRAINT case_type_stages_case_type_id_fkey FOREIGN KEY (case_type_id) REFERENCES public.case_types(id) ON
DELETE CASCADE;


--
-- Name: employees employees_user_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.employees
    ADD CONSTRAINT employees_user_id_fkey FOREIGN KEY (user_id) REFERENCES public.users(id);


--
-- Name: onboarding_tokens onboarding_tokens_user_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
```

```sql
--

ALTER TABLE ONLY public.onboarding_tokens
    ADD CONSTRAINT onboarding_tokens_user_id_fkey FOREIGN KEY (user_id) REFERENCES public.users(id) ON DELETE CASCADE;


--
-- Name: password_reset_tokens password_reset_tokens_user_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
postgres
--

ALTER TABLE ONLY public.password_reset_tokens
    ADD CONSTRAINT password_reset_tokens_user_id_fkey FOREIGN KEY (user_id) REFERENCES public.users(id) ON DELETE
CASCADE;


--
-- Name: project_admin_assignment project_admin_assignment_employee_id_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: postgres
--

ALTER TABLE ONLY public.project_admin_assignment
    ADD CONSTRAINT project_admin_assignment_employee_id_fkey FOREIGN KEY (employee_id) REFERENCES
public.employees(id);


--
-- Name: project_admin_assignment project_admin_assignment_project_id_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: postgres
--

ALTER TABLE ONLY public.project_admin_assignment
    ADD CONSTRAINT project_admin_assignment_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(id) ON
DELETE CASCADE;


--
-- Name: users users_role_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.users
    ADD CONSTRAINT users_role_id_fkey FOREIGN KEY (role_id) REFERENCES public.roles(id);


--
-- Name: work_logs work_logs_case_step_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.work_logs
    ADD CONSTRAINT work_logs_case_step_id_fkey FOREIGN KEY (case_step_id) REFERENCES public.case_steps(id) ON DELETE
CASCADE;


--
-- Name: work_logs work_logs_employee_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: postgres
--

ALTER TABLE ONLY public.work_logs
    ADD CONSTRAINT work_logs_employee_id_fkey FOREIGN KEY (employee_id) REFERENCES public.employees(id)
```

Comunicación entre Componentes

La interacción entre los componentes del sistema sigue las siguientes reglas:

- El **frontend** se comunica únicamente con el **backend** mediante HTTP.
- El **backend** se comunica con la **base de datos** mediante conexiones seguras.
- Los servicios externos (como envío de correos) son accedidos únicamente desde la capa de servicios.
- No existe comunicación directa entre el cliente y la base de datos.

Seguridad y Control de Accesos

La arquitectura lógica contempla mecanismos de seguridad a nivel de aplicación, incluyendo:

- Autenticación de usuarios mediante credenciales cifradas.
- Autorización basada en roles (Admin, Admin de proyecto, desarrollador).
- Protección de endpoints según permisos.
- Manejo de servicios externos mediante credenciales seguras (API Keys).

Beneficios de la Arquitectura Lógica

- **Separación de responsabilidades**, facilitando el mantenimiento del sistema.
- **Escalabilidad**, permitiendo extender servicios sin afectar al frontend.

- **Interoperabilidad**, al utilizar múltiples lenguajes y frameworks.
- **Seguridad**, al centralizar el acceso a datos en la capa de servicios.
- **Flexibilidad**, para futuras integraciones o ampliaciones del sistema.

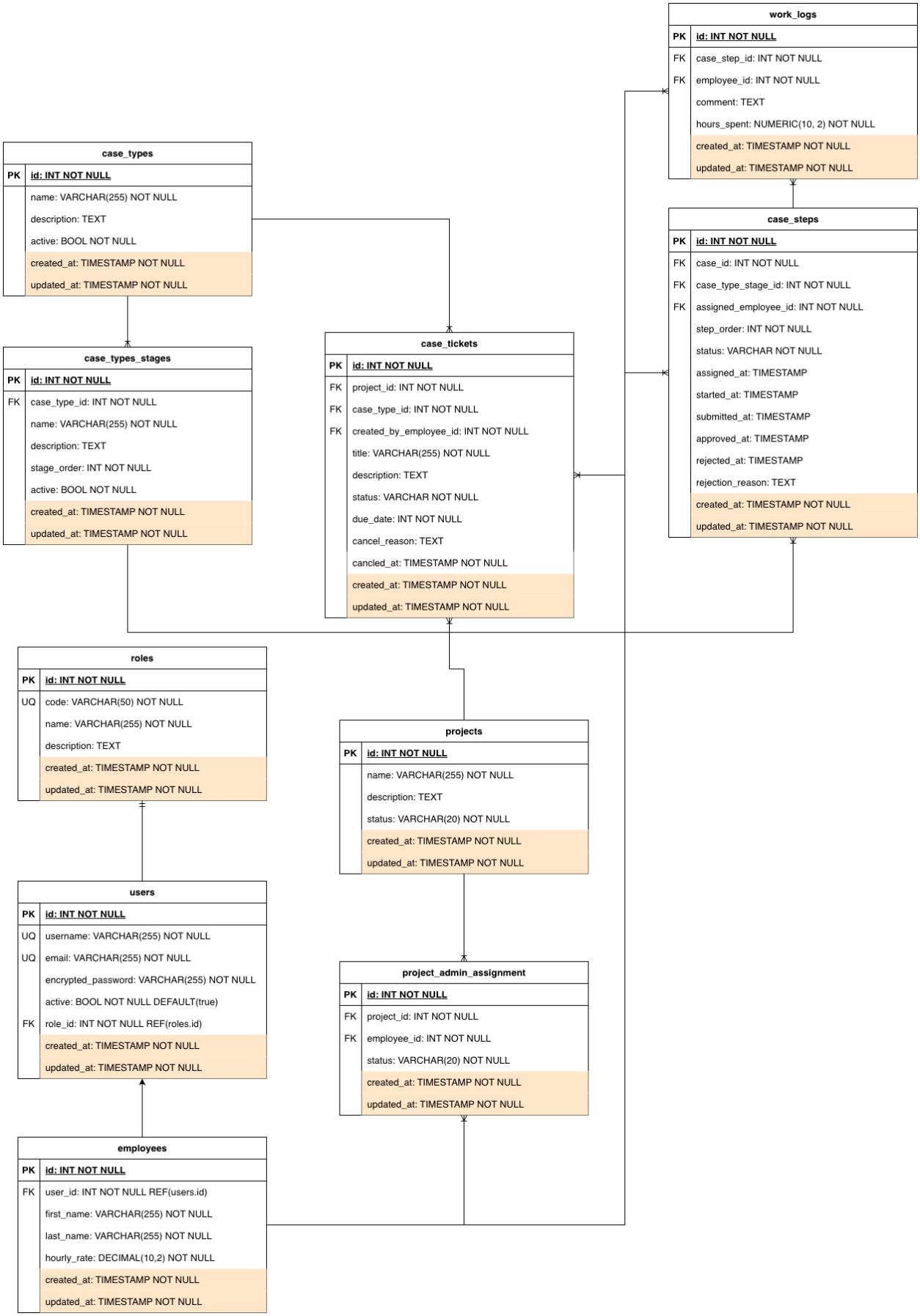



Diagrama Entidad-Relación